Project 3 Descriptions and Resources

Building something completely new does not mean starting from scratch. Often, the most innovative software applications combine existing tools in novel ways.

- This project gives you practice in computer-human interactions to put together and create a new software service.
- You will also get first-hand experience with the agile software design methodology, a popular software development method.
- Finally, you will gain valuable insight into developing accessible software by accounting for multiple accessibility issues.

Project Mission

Scenario

Imagine a venture capitalist approaching you with initial funding to both expand the offerings and improve the user interface of your wildly successful bubble tea shop point-of-sales (POS) system.

- What problems would you attempt to solve?
- How would you improve the frontend of the system?
- How do you maintain the backend of the system?

Objective

Project Solution

In this project, your team will build an integrated web application (i.e., user interface and database system) that blends different existing web components to solve a problem in a novel way for your POS's prototype frontend.

- Such applications are often called hosted or Software as a Service (SaaS).

- This project is open-ended. Your team has the freedom to make many decisions regarding the implementation of the project itself.

Technology Stack

Your project's tech stack will adhere to the following constraints:

- **frontend:** JavaScript or TypeScript; HTML; CSS
- **UI framework:** React or similar
- **backend:** Node.js and Express.js or similar
- **database:** PostgreSQL on TAMU AWS server
- **management:** GitHub Projects or Jira
- **version control:** GitHub in class organization
- **web host:** Render or similar

POS Expectations

Your POS system is expected to adhere to the following:

- The POS system must be for a unique yet imaginary vendor.
  - I.e., the system must not clone an existing vendor.
- The interface design must be unique but may seek limited inspiration from existing other vendors.
  - I.e., the design may take trivial design cues and adjust visual assets of existing bubble tea shop vendors.
- The menu must accommodate a wide range of bubble tea drinks conventionally offered at mainstream bubble tea vendors.
  - I.e., the menu must be able to support different drink customizations and ingredients within reason.

Existing Solutions

Your team is highly encouraged to seek design and menu inspirations from existing bubble tea vendors such as:

  - Sharetea
  - Kung Fu Tea
  - Gong Cha
  - The Alley
  - ChiCha San Chen
  - Wushiland Boba

Focus

A key focus of this project will be the development process, which will include collecting and using user feedback (User Stories and Usability Testing). Your product can utilize different languages behind the scenes, but the overall goal is a solid foundation of functionality and a quality user interface that also includes accessibility elements.

★ Learning Objectives

After successfully completing this project, students will be able to:

- **Apply** the agile development methodology to software design.
- **Design** a publicly hosted web application that provides POS services to multiple types of users.
- **Verify** using a testing methodology, where you will write your test cases before writing your code.
- **Integrate** multiple aspects of accessibility into a software product.
- **Conduct** user studies and adapt a software product based on user feedback.
- **Use** configuration management by creating and using a Git repository for your files.

Project Requirements and Organization

Requirements

Your product must meet the following requirements.

Frontend Development

Develop a web-based application with JavaScript (or TypeScript), HTML, and CSS: The application's primary mode of interaction should be through a browser interface that is written in JavaScript or TypeScript in conjunction with HTML and CSS.

- It must run, at minimum, in Chrome.
- It should not require any extensions to run in the browser.
- It may utilize a user interface (UI) framework such as React or similar.

Backend Development

Host online with PostgreSQL using TAMU AWS: The application must connect to the TAMU AWS server and PostgreSQL database that you used in Project 2.

- You can (and probably should) modify your database's schema and seed data.

- JavaScript-based libraries Node.js and Express.js are the supported technologies in this course for connecting your frontend to your backend.
  - You may alternatively use Python-based (e.g., Django, Flask), Java-based (e.g., Spring Boot), or Ruby-based (e.g., Ruby on Rails) libraries.

Interface Views

Serve different types of users or views: The application should adapt to the different needs of each user type. ***All users are physically in the store, so no mobile app is needed nor allowed.***

- **All Teams**
  - **managers:** desktop conventional interface
    - ***note:*** *required features for your team size are carried over from Project 2*
  - **cashiers:** point-of-sales system interface
    - ***note:*** *required features for your team size are carried over from Project 2*
    - ***note:*** *must be visually and interactively distinct from the customer interface*
  - **customers:** self-service kiosk interface
    - ***note:*** *must be visually and interactively distinct from the cashier interface*
- **Teams of 5 or more**
  - **menu boards:** non-interactive large multi-display interface
- **Teams of 6**
  - **kitchen:** kitchen display interface

Make sure to address the following assumptions for your required interface views:

- **Manager Interface**
  - ***user focus:*** familiar end user
  - ***input modality:*** keyboard-and-mouse input
  - ***place setting:*** privately viewable regular screen in an office room setting
- **Cashier Interface**
  - ***user focus:*** experienced power user
  - ***input modality:*** touchscreen
  - ***place setting:*** privately viewable regular screen behind the restaurant counter
- **Customer Interface**
  - ***user focus:*** casual end user
  - ***input modality:*** touchscreen
  - ***place setting:*** publicly viewable large kiosk of full menu items in the restaurant lobby space
- **Menu Board Interface**
  - ***user focus:*** casual end user
  - ***input modality:*** non-interactive
  - ***place setting:*** publicly viewable large screen of key menu items above the restaurant counter
- **Kitchen Interface**

- ***user focus:*** experienced power user
- ***input modality:*** touchscreen input
- ***place setting:*** semi-publicly viewable regular screen behind the restaurant counter

Make sure to also address the following additional requirements:

- **Portal Page:** all required interfaces must be one-way linked from a singular and centralized portal page.
  - I.e., the portal page must directly link to each required interface, but all the required interfaces must not link back to the portal page.
- **Isolated Access:** the required interfaces must not be accessible to each other.
  - E.g., the customer view must not have an option to log into the manager view.
- **Appropriate Adaptation:** each interface view must be properly adapted to their respective user, input, and place.
  - E.g., the kiosk for the customer view must not assume keyboard-and-mouse input found for a computer in the manager view.

External APIs

Make use of external APIs: The application must utilize the following required external APIs to expand functionality.

- **All Teams**
  - **User Authentication:** A web service that performs user authentication.
    - E.g., **Google OAuth2** (https://developers.google.com/identity/protocols/oauth2), **GitHub OAuth** (https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/creating-an-oauth-app)
    - **Note:** make sure that your application can accommodate logging in from the following email address: **reveille.bubbletea@gmail.com** (mailto:reveille.bubbletea@gmail.com).
  - **Machine Translation:** A web service that performs translation.
    - E.g., **Google Translate** (https://www.w3schools.com/howto/howto_google_translate.asp), **DeepL API** (https://developers.deepl.com/docs/getting-started/intro), **Lara Translation** (https://support.laratranslate.com/en//translation-api).
  - **Weather Forecast:** A web service that provides weather data.
    - E.g., **OpenWeather** (https://openweathermap.org/), **Open-Mateo** (https://open-meteo.com/)
- **Teams of 4 or more**
  - **Personal Assistant:** A web service that provides automated chatbot assistance on menu information and ordering.
    - E.g., **ChatGPT** (https://chatgpt.com/), **Claude** (http://claude.ai/), **Gemini** (https://gemini.google.com/)
- **Teams of 6**

- **Team's Choice:** A web service that is distinct from the required ones that is chosen by the team.
  - Note: must first be approved by the instructor.

Note: With the exception of ***Personal Assistant***, GenAI APIs are not allowed to be used to fulfill required external API functionality.

Web Accessibility

Address accessibility by complying with WCAG 2.1 (https://www.w3.org/WAI/standards-guidelines/wcag/) and the needs of the given persona in your design: You can refer to accessibility resources such as **Introduction to Web Accessibility (https://www.w3.org/WAI/fundamentals/accessibility-intro/)**.

- It is industry standard for the entire system to be accessibility compliant.
- Your customer interface at the minimum must be compliant for all your required personas.
- Refer to **Project 3 Accessibility Personas (https://canvas.tamu.edu/courses/434394/pages/project-3-accessibility-personas)** for further requirements details.

Online Deployment

Deploy and (remain deployed) by the end of Sprint 1: The application must stay deployed and be updated regularly at all times thereafter.

- At any point during Project 3, failure to have your MVP deployed and updated may result in penalizations to your subsequent GitHub releases.

Version Control

Utilize GitHub to version control your repository: The application code must utilize version control using GitHub in your project workflow.

- The repository must be hosted in the GitHub class organization.

Above-and-Beyond Functionality

Go beyond the above requirements: The application must support non-trivial thematic functionality that extends beyond a straightforward restaurant POS system (i.e., ***beyond features***).

- Designing and developing this thematic functionality will require additional thought, features, and implementation.

- For each team with *n* members, that team must support (*n* - 3) different beyond features for the customer interface.
  - Consult your teaching assistant for each of your proposed beyond features to determine if they are legitimately beyond the above requirements.
  - For teams of 6 that plan to use their *Team's Choice* API for one of their beyond features, ensure that the beyond feature is not just a trivial implementation of the *Team's Choice* API.
    - That is, the *Team's Choice* API must support the functionality of the beyond feature, not be the entire beyond feature (i.e., no "double-dipping").
  - Functionality that is a beyond feature must be unique from and cannot also be counted as an external API (i.e., no "double-dipping").

Presentation and Demo

Your final product will be presented at the end of the semester (see **Finals Location and Times (https://aggie.tamu.edu/registration-and-records/classes/final-examination-schedules)**).

- The final time associated with the lecture will be when you present your working product to the instructor, TAs, and other classmates.
  - The goals of this presentation are to show the evolution of your product from the initial design to the currently deployed service and to convince those funding you to continue.
- The final time you associated with your lab will be a technical demonstration with questions on back-end development to your TA.
  - The demo's goals are to verify that your application meets the assignment's requirements and assess the degree to which those requirements are met or exceeded.

Software Development Life Cycle

You will use an Agile SDLC for this project.

- Due to team size and, even more significantly, time constraints, we will not be able to follow a "real" agile approach, but we will try to capture some of the main themes from an Agile development process using Scrum.
- Time management will be crucial to your success. Requirement changes from your initial design will come from the user studies that you provide.

Approach

We will use an iterative approach to this assignment.

- Your team will be asked to implement the program over three "sprints," each one week in length.

- Features and functionality (i.e., "value") should be added during each Sprint.
- At the end of each Sprint, your team should have working software.
- Between each Sprint is a 1-week reflection and planning period.
- In addition, you are to make backlogs, create burn-down charts, and hold Scrum meetings.

Management

We will use project managers in this assignment.

- Your team will be asked to hold SCRUM meetings with a chosen SCRUM manager at least three (3) times per sprint.
- These meetings are approximately fifteen (15) minutes each for your team to share what is working and not working with each other.

Grading

Project Deliverables

The following assignments are the project deliverables that directly relate to your technical contributions of the project:

1. **Sprint 1**
    1. Project 3 Deliverable: Sprint 1 GitHub Release [Team]
    2. Project 3 Deliverable: Sprint 1 Written Materials [Team]
    3. Project 3 Deliverable: Project 3 User Study Report [Team]
2. **Sprint 2**
    1. Project 3 Deliverable: Sprint 2 GitHub Release [Team]
    2. Project 3 Deliverable: Sprint 2 Written Materials [Team]
    3. Project 3 Deliverable: Mid-Project Presentation Slides [Team]
3. **Sprint 3**
    1. Project 3 Deliverable: Sprint 3 GitHub Release [Team]
    2. Project 3 Deliverable: Sprint 3 Written Materials [Team]
4. **End-Project**
    1. Project 3 Deliverable: Final Comprehensive Demo [Team]
    2. Project 3 Deliverable: Final Presentation [Team]
    3. Project 3 Deliverable: End-Project Team Retrospective [Team]
    4. Project 3 Deliverable: Final Examination Demo [Team]

Written Elements

The following assignment is a written elements that directly relate to your written requirements for the project:

1. Written Elements: Project 3 Retrospective Report [Individual]

Peer Evaluations

The following assignments are the peer evaluations that indirectly contextualize your individual performance in the project:

1. Peer Evaluation: Project 3 Start
2. Peer Evaluation: Project 3 Midpoint
3. Peer Evaluation: Project 3 End

Reminder: *As stated in the syllabus, assignments in the Peer Evaluations category are not eligible for late submission.*

Individual Performance

The following assignments are the individual performance checks that directly relate to your individual performance in the project:

1. Individual Performance: Project 3 Start
2. Individual Performance: Project 3 Midpoint
3. Individual Performance: Project 3 End

Note: *Assignments in the Individual Performance category are non-submissions and are instead graded by the instructor and teaching assistants based on various assessment sources.*

Timeline for Deliverables

1st Week (Week 10)

- Identify and explore issues with your current user interface.
- Investigate deployed interfaces in real-world settings.

2nd Week (Week 11)

- Plan and start Sprint 1.

3rd Week (Week 12)

- Complete Sprint 1 and release Sprint 1 MVP.
- Conduct a user study on your Sprint 2 prototype, write your user study report.
- Evaluate your peers.

4th Week (Week 13)

- Plan and start Sprint 2.
- Report user study findings on your most recent MVP.

5th Week (Week 14)

- Complete Sprint 2 and release Sprint 2 MVP.
- Evaluate your peers.

6th Week (Week 15)

- Plan and start Sprint 3.
- Complete mid-project presentation slides.

7th Week (Week 16)

- Complete Sprint 3 and release Sprint 3 MVP.
- Evaluate your peers.
- Conduct your end-project team retrospective.
- Write an end-project individual retrospective.

Final Exam Week

- Present and demo project at the end of the semester.

A Word on Academic Integrity

Team Assignment

This project is a team assignment.

- Your work should be completed only within your team.
- You should not use any work or designs (regardless of your access) from any other team or anyone outside this class.

- You should protect your team's work by making sure that online collaboration materials are kept private within the team.

Helpful Resources

The subsequent section contains some helpful resources and tutorials for your project.

- You may use other online tutorials, but you may not use someone else's code, whether they are associated with this class or not.
- If you are unsure whether a resource is approved, assume it is not and ask your instructor or TA.

Individual Contributions

Finally, you will be asked to describe your individual contribution and the contributions of your team members.

- Remember that the **Aggie Honor Code applies (https://student-rules.tamu.edu/aggiecode/)** here as well.
- You must accurately and objectively describe these contributions.

Helpful Tips and Resources.

APIs

- **GitHub API Listing (https://github.com/public-apis/public-apis)**
- **AnyAPI Listing (https://marketplace.apilayer.com/?utm_source=any-api&utm_medium=any-api-redirection&utm_campaign=any-api-redirection)**
- **Another List of Free APIs (https://apilist.fun/)**

GUI Sketching

- **Miro (https://miro.com/)**
- **Sketchboard.io (https://sketchboard.io/)**
- **Figma (https://www.figma.com/)**

Writing and Feedback

- Use a spell checker.
- **Grammarly (https://www.grammarly.com/)** for checking grammar.

- **Hemmingway App** (https://hemingwayapp.com/) for style. Note: you may not want to use all the suggestions, but some are helpful.
- **University Writing Center** (https://writingcenter.tamu.edu/) assists students at any stage of the writing process from brainstorming to proofreading
  - Set up an **appointment to get feedback** (https://cleopatra.tamu.edu/default.aspx?ReturnUrl=%2f) from a UWC consultant via Zoom or online